

Screen Elophant – Release- & Update-Prozess

Standard-Richtlinie & Engineering Guide • Stand: 2026

Diese Richtlinie definiert den verbindlichen Deployment- und Release-Zyklus für die Bühnenpräsentations-Software **Screen Elophant**. Die Einhaltung dieser Sequenz sichert die Integrität der Versionsstände und den automatisierten Build-Flow von Elophant Labs.

◆ 1. Versions-Definition

Sämtliche Versionierungen folgen strikt den Vorgaben von **Semantic Versioning 2.0.0** (Muster: `x.x.x`):

- **Major (x.0.0):** Breaking Changes oder fundamentale Architektur-Updates
- **Minor (0.x.0):** Abwärtskompatible funktionale Erweiterungen und neue Features
- **Patch (0.0.x):** Abwärtskompatible Bugfixes, Hotfixes und Performance-Optimierungen

◆ 2. Schritt-für-Schritt Release-Anleitung

2.1 Änderungen stagen

Überführung aller finalisierten Quellcodedateien und Assets in den Git-Staging-Bereich:

```
git add .
```

2.2 Build-Commit erstellen

Verbindliches Benennungsmuster für den Vorab-Commit verwenden: `-m "build vx.x.x"`.

```
git commit -m "build v0.4.3"
```

2.3 Push in den Entwicklungs-Branch

Aktualisierung des Remote-Repositories mit dem aktuellen Build-Stand:

```
git push
```

2.4 Automatisiertes Tagging via npm

Generierung der neuen Version. Durch diesen Befehl wird automatisch ein nativer Git-Tag injiziert.

```
npm version 0.4.3
```

⚙️ ABGLEICH DURCH NPM VERSION

Der Workspace-Abgleich modifiziert die Versions-Strings vollautomatisch an exakt **7 Systemstellen**:

- `package.json` (1x)
- `package-lock.json` (1x)
- `tauri.conf.json` (1x)
- `StartingWindow/index.html` (1x)
- `Updater.json` (3x)

2.5 Build-Trigger & Tag-Push

Pushen des generierten Commits inklusive aller Tags, um die CI/CD-Pipeline zu initiieren:

```
git push --follow-tags
```

Hinweis: Erst durch das Flag --follow-tags detektieren die GitHub Actions den Release-Tag und starten den Compile-Prozess.

2.6 Public Signature extrahieren

- Navigiere auf GitHub in das Tab **Actions** des Screen-Ellophant-Repositories.
- Wähle den durch den Tag-Push getriggerten Build-Workflow aus.
- Kopiere die verschlüsselte **Public Signature** aus der Konsolenausgabe des Build-Logs.

2.7 Updater-Konfiguration aktualisieren

Trage die kopierte Public Signature in die lokale Datei `update.json` ein, um den Client-seitigen Autoupdater zu autorisieren.

2.8 Finalen Release versiegeln

Abschluss des Release-Prozesses durch Stagen der signierten Konfiguration und finalen Push:

```
git add .  
git commit -m "release v0.4.3"  
git push
```

Wichtig: Ersetze das v0.4.3-Muster stets durch die jeweils aktuell vergebene Versionsnummer.

Dokumenten-ID: EL-SE-REL-2026 • Vertraulichkeit: Intern • Ellophant Labs Engineering